INSTITUT TEKNOLOGI TANGERANG SELATAN

Mata Kuliah - Semester 1 - Teknik Informatika

IF104

Pemrograman Berorientasi Objek

Minggu ke-2: Prinsip Dasar OOP

IF104 - 3 SKS | Semester Ganjil 2026/2027

Dhika Anbiya

CAPAIAN PEMBELAJARAN

Capaian Pembelajaran Minggu Ini

CPL LULUSAN

CPL-03, CPL-04, CPL-08, CPL-09

CPMK MATA KULIAH

CPMK-IF104-1 - Menjelaskan konsep OOP

SUB-CPMK MINGGU 2

SUB-CPMK-IF104-1-2

Mahasiswa mampu menguraikan prinsip dasar OOP (enkapsulasi, inheritance, polimorfisme, abstraksi) secara tepat berdasarkan kriteria yang ditetapkan.

AGENDA

Agenda Pertemuan Minggu ke-2

1 **Enkapsulasi**

Konsep dan implementasi: menyembunyikan data dengan access modifier.

2 **Inheritance**

Hierarki kelas dan pewarisan atribut serta metode.

3 **Polimorfisme**

Overloading dan overriding: satu nama, banyak bentuk.

4 **Abstraksi**

Kelas abstrak dan antarmuka: menyembunyikan kompleksitas.

Setiap prinsip diikuti contoh program dalam Java dan C++.

REVIEW

Dari Kelas dan Objek ke Empat Pilar

MINGGU 1

Kelas dan Objek

Program disusun dari kelas (cetak biru) dan objek (instance). Objek menyimpan data dan perilaku.

```
class Mobil { ... }
```

1. Enkapsulasi
2. Inheritance
3. Polimorfisme
4. Abstraksi

MINGGU 2

SUBTOPIK 1

Enkapsulasi: Membungkus Data

Enkapsulasi adalah prinsip membungkus data (atribut) dan perilaku (metode) ke dalam satu unit, lalu menyembunyikan detail internalnya. Akses dari luar hanya lewat metode yang disediakan.

Sembunyikan data

Atribut dibuat private agar tidak diubah sembarangan.

Akses terkontrol

Nilai dibaca lewat getter dan ditulis lewat setter.

Validasi masuk

Setter memeriksa input sebelum data disimpan.

Access modifier: private (dalam kelas) - public (terbuka) - protected (pewarisan).

SUBTOPIK 1

Enkapsulasi - Java

Rekening.java

```java
public class Rekening {
    private double saldo;        // data tersembunyi

    public double getSaldo() {   // getter
        return saldo;
    }

    public void setSaldo(double s) { // setter
        if (s >= 0) saldo = s;    // validasi
    }
}
```

INTI

Private + Getter/Setter

private saldo

Atribut disembunyikan; hanya diakses dari dalam kelas.

Getter getSaldo()

Membaca nilai tanpa membuka atribut langsung.

Setter setSaldo()

Menulis nilai dengan validasi (s >= 0).

SUBTOPIK 1

Enkapsulasi - C++

rekening.cpp

```
class Rekening {
private:
    double saldo;                // data tersembunyi
public:
    double getSaldo() {          // getter
        return saldo;
    }
    void setSaldo(double s) {    // setter
        if (s >= 0) saldo = s;   // validasi
    }
};
```

INTI

Bedanya dengan Java

Label akses

private: / public: adalah label bagian, bukan per anggota.

Getter & setter

Konsep sama dengan Java; penempatan label yang berbeda.

Titik koma

Akhir definisi class C++ ditutup dengan "};".

SUBTOPIK 2

Inheritance: Pewarisan Kelas

Inheritance (pewarisan) memungkinkan kelas baru mewarisi atribut dan metode dari kelas induk (superclass). Relasinya disebut "is-a": Mobil adalah Kendaraan.

Superclass

Kelas induk: berisi atribut dan metode yang umum.

Subclass

Kelas anak: mewarisi dan menambah anggota sendiri.

Relasi is-a

Subclass adalah jenis khusus dari superclass.

Java: class Mobil extends Kendaraan | C++: class Mobil : public Kendaraan

SUBTOPIK 2

Inheritance - Java

Kendaraan.java

```java
class Kendaraan {
    String merek;
    void maju() {
        System.out.println("maju");
    }
}

class Mobil extends Kendaraan { // is-a
    int jumlahPintu;
}
```

INTI

Keyword extends

extends

Mobil mewarisi merek dan maju() dari Kendaraan.

Anggota baru

Mobil menambah jumlahPintu sendiri.

Tanpa duplikasi

Kode umum cukup ditulis sekali di superclass.

SUBTOPIK 2

Inheritance - C++

kendaraan.cpp

```cpp
class Kendaraan {
public:
    std::string merek;
    void maju() {
        std::cout << "maju";
    }
};

class Mobil : public Kendaraan { // pewarisan
public:
    int jumlahPintu;
};
```

INTI

Bedanya dengan Java

Tanda : public

C++ memakai ":" bukan "extends".

Mode pewarisan

public menjaga akses anggota superclass.

Titik koma

Setiap class C++ ditutup "};".

SUBTOPIK 3

Polimorfisme: Satu Nama, Banyak Bentuk

Polimorfisme berarti satu nama metode dapat memiliki banyak bentuk. Ada dua jenis: overloading (waktu kompilasi) dan overriding (waktu berjalan).

Overloading

Nama sama, parameter beda. Diputuskan saat kompilasi.

Overriding

Subclass menimpa metode superclass. Diputuskan saat berjalan.

Overloading = compile-time | Overriding = runtime

SUBTOPIK 3

Overloading - Java

Hitung.java

```java
class Hitung {
    int tambah(int a, int b) {
        return a + b;
    }
    double tambah(double a, double b) {
        return a + b;
    }
    int tambah(int a, int b, int c) {
        return a + b + c;
    }
}
```

INTI

Nama sama, parameter beda

Parameter beda

Dibedakan oleh jumlah atau tipe parameter.

Compile-time

Pilihan metode ditentukan saat kompilasi.

Return beda

Tipe kembalian boleh berbeda selama parameter beda.

SUBTOPIK 3

Overloading - C++

hitung.cpp

```cpp
class Hitung {
public:
    int tambah(int a, int b) {
        return a + b;
    }
    double tambah(double a, double b) {
        return a + b;
    }
    int tambah(int a, int b, int c) {
        return a + b + c;
    }
};
```

INTI

Bedanya dengan Java

Aturan sama

Parameter beda = metode beda, identik dengan Java.

Overload operator

C++ juga bisa me-overload operator (+, -).

Titik koma

Akhir class memakai "};".

SUBTOPIK 3

Overriding - Java

Hewan.java

```java
class Hewan {
    void suara() {
        System.out.println("...");
    }
}

class Kucing extends Hewan {
    @Override
    void suara() {
        System.out.println("meong");
    }
}
```

INTI

Menimpa metode superclass

Tanda @Override

Menandai metode yang menimpa superclass.

Runtime

Versi yang dipanggil ditentukan saat berjalan.

Subclass

Kucing mengganti perilaku suara() dari Hewan.

SUBTOPIK 3

Overriding - C++

hewan.cpp

```cpp
class Hewan {
public:
    virtual void suara() {
        std::cout << "...";
    }
};

class Kucing : public Hewan {
public:
    void suara() override {
        std::cout << "meong";
    }
};
```

INTI

Bedanya dengan Java

Wajib virtual

Metode superclass harus virtual agar bisa ditimpa.

Kata override

C++11 memakai "override" sebagai penanda.

Tanpa @

Tidak memakai "@"; cukup tulis ulang metode.

SUBTOPIK 4

Abstraksi: Menyembunyikan Kompleksitas

Abstraksi menampilkan hal yang penting dan menyembunyikan detail implementasi. Di OOP, abstraksi diwujudkan lewat kelas abstrak dan antarmuka (interface).

Kelas abstrak

Kelas yang tidak bisa dibuat objek; punya method abstrak tanpa isi.

Antarmuka

Kontrak berisi method yang wajib diimplementasikan kelas pemakainya.

abstract class / interface (Java) | pure virtual (C++)

SUBTOPIK 4

Kelas Abstrak - Java

Bentuk.java

```
abstract class Bentuk {
    abstract double luas(); // tanpa isi
}

class Lingkaran extends Bentuk {
    double r;
    double luas() {
        return 3.14 * r * r;
    }
}
```

INTI

abstract class

Tidak bisa new

Bentuk tidak bisa dibuat objeknya.

Method abstrak

luas() wajib diimplementasi subclass.

Subclass konkret

Lingkaran mengisi implementasi luas().

SUBTOPIK 4

Kelas Abstrak - C++

bentuk.cpp

```cpp
class Bentuk {
public:
    virtual double luas() = 0; // pure virtual
};

class Lingkaran : public Bentuk {
public:
    double r;
    double luas() override {
        return 3.14 * r * r;
    }
};
```

INTI

Bedanya dengan Java

Pure virtual

"= 0" menjadikan kelas abstrak.

Tidak ada keyword

Tidak memakai "abstract"; cukup method = 0.

Wajib override

Subclass harus mengisi luas().

SUBTOPIK 4

Antarmuka (interface) - Java

BisaTerbang.java

```java
interface BisaTerbang {
    void terbang();         // method abstrak
}

class Burung implements BisaTerbang {
    public void terbang() {
        System.out.println("terbang");
    }
}
```

INTI

interface + implements

Kontrak

Interface mendefinisikan method yang wajib ada.

implements

Kelas mewujudkan kontrak dengan implements.

Banyak interface

Satu kelas boleh implements beberapa interface.

SUBTOPIK 4

Antarmuka - C++

bisaterbang.cpp

```cpp
class BisaTerbang {
public:
    virtual void terbang() = 0; // antarmuka
};

class Burung : public BisaTerbang {
public:
    void terbang() override {
        std::cout << "terbang";
    }
};
```

INTI

Bedanya dengan Java

Kelas abstrak murni

C++ memakai kelas semua-method = 0.

Tidak ada interface

C++ tidak punya keyword interface.

Warisan ganda

C++ mendukung pewarisan dari banyak kelas.

RINGKASAN

Java vs C++: Sintaks Kunci

Aspek	Java	C++
Pewarisan	`class B extends A`	`class B : public A`
Overriding	`@Override`	`virtual + override`
Antarmuka	`interface + implements`	`class murni (method = 0)`
Kelas abstrak	`abstract class`	`pure virtual (= 0)`
Enkapsulasi	`private per anggota`	`label private: / public:`
Akhir kelas	`}`	`};`
Tipe string	`String`	`std::string`

Konsep OOP sama; yang berbeda hanya cara menulisnya.

LATIHAN

Analisis Kode: Temukan Pilarnya

Latihan.java

```java
class Hewan {
    void suara() {
        System.out.println("...");
    }
}

class Kucing extends Hewan {
    void suara() {          // menimpa
        System.out.println("meong");
    }
    void suara(int kali) {  // beban lebih
        for (int i = 0; i < kali; i++)
            System.out.println("meong");
    }
}
```

DISKUSI

Tiga pertanyaan

1. Inheritance

Di baris mana pewarisan terjadi?

2. Overriding

Metode mana yang menimpa superclass?

3. Overloading

Metode mana yang overload (parameter beda)?

SELANJUTNYA

Minggu ke-3: Karakteristik dan Perbandingan OOP

1. Perbandingan OOP dengan pemrograman prosedural
2. Keuntungan dan kerugian OOP
3. Studi kasus: perangkat lunak skala besar
4. Evaluasi pemilihan paradigma

Ada Kuis Minggu ke-3 (bobot 5%). Persiapkan perbandingan OOP vs prosedural.

RANGKUMAN

Yang Perlu Diingat Hari Ini

Enkapsulasi	Bungkus data dengan private; akses lewat getter dan setter.
Inheritance	Kelas anak mewarisi anggota kelas induk (extends / : public).
Polimorfisme	Overloading (compile-time) dan overriding (runtime).
Abstraksi	Kelas abstrak dan antarmuka menyembunyikan detail.

TERIMA KASIH

Pemrograman Berorientasi Objek - Minggu ke-2: Prinsip Dasar OOP

REFERENSI

[1] Deitel, P. & Deitel, H. (2017). Java: How to Program.

[2] Horstmann, C. S. (2019). Core Java, Vol. I.

[3] Oracle. The Java Tutorials.

[4] JetBrains. IntelliJ IDEA Documentation.